

NCBA LOOP CASE STUDY

Country Integration Service

Integrations and Microservices Engineer

Samuel Mutua Kimani
8 October 2026

Component responsibilities

REST boundary

CountryController accepts JSON. Request DTOs validate fields. Exception advice returns clear HTTP errors.

Application logic

CountryService normalizes names, checks duplicates, and coordinates SOAP calls with persistence.

Integration and storage

CountrySoapClient parses SOAP results. SoapTransport handles HTTP failures. JPA repositories access MySQL.

SOAP integration flow

1. Resolve the ISO code

POST /api/countries accepts {"name":"kenya"}. The service converts the name to Kenya and calls CountryISOCode.

2. Retrieve country details

The returned KE code becomes the input to FullCountryInfo. Namespace-aware parsing maps country fields and languages.

3. Persist and respond

MySQL stores the country and languages. The endpoint returns 201 Created with a generated ID and Location header.

Data model and REST operations

CountryInfo and Language

CountryInfo stores ISO code, name, capital, phone, continent, currency, and flag URL. Language rows reference their country.

CRUD endpoints

POST creates. GET lists or retrieves by ID. PUT updates editable details and replaces languages. DELETE returns 204.

Consistency

Unique country ISO codes prevent duplicates. Update and delete use transactions. SOAP calls occur before the database save.

Failure handling

Timeouts and retries

HTTP connect timeout: 5 seconds. Request timeout: 8 seconds. Eligible failures receive up to 2 attempts, 300 ms apart.

Circuit breaker

10-call window, minimum 5 calls, 50% failure threshold. Open duration: 30 seconds. Half-open permits 2 calls.

Degraded operation

Creation returns an explicit upstream error. Stored-country reads continue through MySQL while SOAP is unavailable.

HTTP responses

400 validation, 404 missing country, 405 method, 409 duplicate, 502 upstream response, 503 unavailable, 504 timeout.

Logs and health checks

Request visibility

ECS JSON logs include request ID, method, path, HTTP status, and duration. Responses expose X-Request-ID.

Integration diagnostics

SOAP logs record retry attempts, failures, and circuit transitions. Actuator exposes HTTP request timing metrics.

Operational health

Startup allows ten minutes; probes time out after ten seconds. Readiness checks MySQL; six failed liveness checks trigger a restart.

27 tests passed

Service behavior, 10 tests

Creation and normalization, duplicate handling, update/delete behavior, and preservation of upstream error statuses.

SOAP parsing, 7 tests

Namespaces and full details, unknown country, SOAP faults, malformed XML, DOCTYPE rejection, and XML escaping.

REST and transport, 9 tests

6 controller tests cover responses and validation. 3 transport tests verify retries, non-retryable errors, and circuit opening.

Application context, 1 test

The full Maven package run connected Spring Boot to MySQL. Result: 27 tests, 0 failures, 0 errors, 0 skipped.

Container and Kubernetes deployment

Docker Compose

Java 25 runtime image runs as a non-root user. Application connects to mysql:3306. The MySQL named volume preserves data.

Kubernetes resources

ConfigMap, credential Secret, application Deployment and Service. MySQL uses one Deployment and a 2 GiB storage claim.

Deployment verification

Recovered after CPU/probe tuning: both replicas ready, health and data checks passed, zero restarts. Startup: 64 and 49 seconds. Production availability unverified.

Shared data across two replicas

Write through replica one

POST Tanzania returned 201 Created and Location /api/countries/2. MySQL persisted the country and language.

Read through replica two

GET countries returned 200 and included Kenya and Tanzania, confirming shared persistence between application replicas.

Scale boundary

During the successful test, the Service had two ready backends. Both replicas ran on one local node; multi-node availability was not demonstrated.

Design decisions and production work

MySQL and stateless replicas

Shared durable storage keeps countries consistent across application instances. A single database simplifies this assessment deployment.

Synchronous country creation

Direct SOAP calls return details immediately. Retries and timeouts bound failure handling. Queues would suit a future bulk import workflow.

High-load direction

Measure load before tuning replicas and database pools. Add paging and cache frequently read country data with explicit expiry and invalidation.

Production improvements

Versioned migrations, isolated persistence tests, TLS and access controls, database failover, backups, and monitored capacity remain future work.

Run and troubleshoot

Build and run

Start MySQL, export DB_PASSWORD, run ./mvnw clean package, then docker compose up -d --build. Local API: localhost:8081.

Deploy to Kubernetes

Create namespace and Secret, apply MySQL and app manifests, inspect rollout and PVC, then port-forward the Service to localhost:8082.

Troubleshooting guide

Inspect logs and pod events. Check image availability, ports, credentials, storage binding, readiness, and resource limits.

Submission contents

<https://github.com/kimtour/country-integration-service>
Source, tests, README and deployment guide included.